Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Владимирский государственный университет
имени Александра Григорьевича и Николая Григорьевича Столетовых»
(ВлГУ)
Кафедра физики и прикладной математики

ЛАБОРАТОРНАЯ РАБОТА № 1

по дисциплине: «Технологии разработки веб-приложений»
на тему: «Проектирование архитектуры и макетов веб-приложения»

Выполнили:
студенты группы ИТ-124
Володина Софья Дмитриевна
Горностаев Александр Александрович

Проверил:
доцент каф. ФиПМ
Лексин Андрей Юрьевич

Владимир, 2026

Цель: развитие навыков разработки и согласования технических спецификаций на веб-приложения.

Задание: создать рабочее пространство для выполнения лабораторных работ и курсовой работы. Спроектировать и задокументировать информационную и программную архитектуру веб приложения, разработать макеты страниц.

Практическая часть.

Курсовая работа посвящена разработке веб-приложения «Игровая энциклопедия». Проект представляет собой сайт-каталог, в котором пользователи могут просматривать сведения об играх, управлять личным профилем, оставлять отзывы и сохранять материалы в закладки.

Для выполнения лабораторных и курсовой работы был создан **Git-репозиторий**, обеспечивающий командную разработку и контроль версий:

Ссылка на репозиторий: https://github.com/Alex-c18/game-encyclopedia

В репозитории предусмотрена отдельная папка /docs для хранения технических спецификаций, UML-диаграмм и макетов интерфейса. Также сформирован файл .gitignore для исключения служебных файлов IDE, временных кэшей, зависимостей (node_modules/) и скомпилированных бинарных файлов (bin/, obj/).

Разработана техническая документация (README.md), включающая описание предметной области, перечень технологического стека, инструкции по локальному развёртыванию и описание структуры каталогов.

Спроектирована файловая структура проекта:

/src/server – исходный код серверного приложения;

/src/client – исходный код клиентского приложения;

/docs – спецификации, UML-диаграммы и макеты интерфейса.

Игровая индустрия и компьютерные игры являются **предметной областью проекта**. «Игровая энциклопедия» создана для систематизации сведений об играх и формирования удобного пользовательского каталога, дополненного расширенными функциями поиска, фильтрации и персонализации.

Приложение рассчитано на широкую аудиторию игроков: от пользователей, которым требуется информация об играх, до зарегистрированных участников, способных формировать собственный список закладок, размещать отзывы и редактировать профиль. Администратор выполняет отдельную функцию: следит за актуальностью каталога и контролирует пользовательские материалы. К основным возможностям относятся:

1) Гостевой пользователь:

– просмотр каталога игр с фильтрацией (по жанрам, году выпуска, платформам) и сортировкой;

– поиск игр по названию;

– просмотр детальной карточки игры (описание, системные требования, медиагалерея, внешние ссылки на Steam/официальный сайт);

– просмотр отзывов и комментариев пользователей;

– регистрация и аутентификация в системе.

2) Авторизованный пользователь (наследует возможности гостя):

– добавление и удаление игр из личного списка («Закладки»);

– публикация комментариев и оценок на страницах игр;

– управление персональным профилем.

3) Администратор:

– управление каталогом (добавление, редактирование, удаление игр);

– модерация пользовательских отзывов и комментариев.

Функционал приложения графически представлен в виде **UML-диаграммы** прецедентов. На рисунке 1 отражены три роли: Гость,

Авторизованный пользователь и Администратор, а также соответствующие им прецеденты.

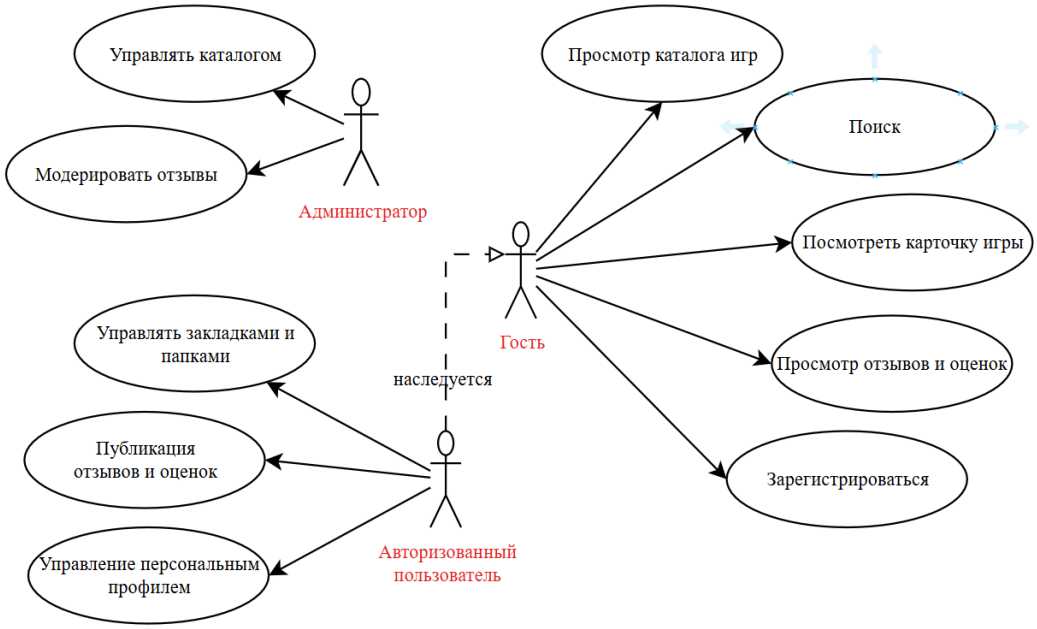


Рисунок 1 – UML-диаграмма прецедентов

В основе **информационной структуры** лежит главная страница, которая является точкой входа и обеспечивает навигацию по основным разделам приложения.

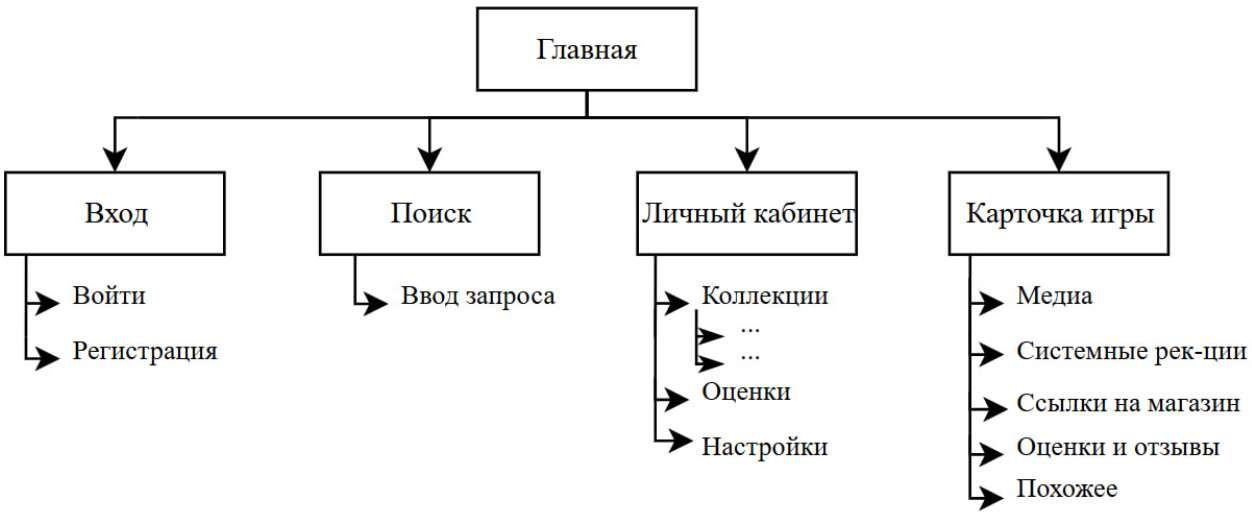


Рисунок 2 – информационная структура

От главной страницы отходят четыре ключевых узла:

1) Вход. Раздел аутентификации и регистрации пользователей. Содержит два дочерних элемента:

a. войти – форма авторизации для существующих пользователей;

b. регистрация – форма создания нового аккаунта.

2) Поиск. Раздел поиска игр по названию. Содержит дочерний элемент:

a. ввод запроса – поле ввода поискового запроса с последующей выдачей результатов.

3) Личный кабинет. Закрытый раздел для авторизованных пользователей. Содержит три дочерних элемента:

a. коллекции – список сохранённых игр (закладок) с возможностью группировки по папкам;

b. оценки – список выставленных пользователем оценок и отзывов;

c. настройки – управление персональным профилем (изменение данных, аватар, пароль).

4) Карточка игры. Детальная страница конкретной игры. Содержит пять дочерних элементов:

a. медиа – галерея скриншотов, трейлеры, обложки;

b. системные требования – минимальные и рекомендуемые характеристики ПК;

c. ссылки на магазины – внешние ссылки на Steam, официальный сайт и другие платформы;

d. оценки и отзывы – пользовательские оценки и комментарии к игре;

e. похожее – блок рекомендаций с похожими играми.

Такая организация делает навигацию понятной и удобной: с главной страницы можно открыть поиск, выполнить авторизацию, перейти в личный кабинет или просмотреть страницу игры. В карточке игры размещены материалы, системные требования, ссылки на магазины, отзывы и рекомендации. Доступ к личному кабинету предоставляется после авторизации; в нём собраны коллекции, оценки и настройки профиля.

При проектировании веб-приложения «Игровая энциклопедия» выбор **технологического стека** производился с учётом требований к производительности, скорости разработки, строгости типизации и удобству командной работы.

1. Серверная часть и база данных (Backend & Database)

ASP.NET Core Web API (C#)

– Высокая производительность: фреймворк ASP.NET Core и встроенный веб-сервер Kestrel входят в число лидеров по скорости обработки асинхронных HTTP-запросов.

– Архитектура и архитектурные паттерны: встроенный механизм внедрения зависимостей (Dependency Injection), удобная реализация REST API через контроллеры и middleware-компоненты для обработки ошибок и авторизации.

– Безопасность: простая интеграция аутентификации и авторизации на базе JWT-токенов и хеширования паролей.

Entity Framework Core (EF Core)

– Безопасность типов и LINQ: использование C#-запросов LINQ вместо чистого SQL исключает ошибки в синтаксисе на этапе компиляции и защищает приложение от SQL-инъекций.

– Подход Code-First и миграции: позволяет проектировать структуру базы данных через C#-модели. Механизм миграций автоматически отслеживает изменения и обновляет схему БД, что критично при командной разработке.

PostgreSQL

– Надёжность и ACID-соответствие: реляционная СУБД с высокой надёжностью хранения данных и поддержкой сложных транзакций (необходимо для корректного учёта пользователей, отзывов и закладок).

– Поддержка JSON/JSONB: позволяет сочетать реляционную структуру со слабоструктурированными данными (гибкие системные требования игр, дополнительная метаинформация).

– Отличная интеграция с EF Core: официальный провайдер Npgsql.EntityFrameworkCore.PostgreSQL обеспечивает высокую скорость работы и полную поддержку возможностей PostgreSQL.

2. Клиентская часть (Frontend)

React

– Компонентный подход: интерфейс разбивается на независимые, переиспользуемые блоки (карточка игры, сетка каталога, поисковая строка, модальное окно авторизации), что упрощает поддержку и масштабирование кода.

– Virtual DOM и SPA: обеспечивает мгновенный отклик интерфейса и переключение между страницами без полной перезагрузки браузера.

TypeScript

– Статическая типизация: обнаружение ошибок на этапе написания кода, а не в runtime.

– Синхронизация структур данных: интерфейсы TypeScript на фронтенде точно повторяют DTO-модели C# с бэкенда, предотвращая ошибки при передаче данных по REST API.

– Улучшенный автокомплит и рефакторинг: наличие типов упрощает навигацию по проекту и командную разработку.

Vite

– Высокая скорость сборки: в отличие от устаревшего Create React App (Webpack), Vite использует нативные ES-модули браузера, обеспечивая мгновенный запуск локального сервера разработки и супербыструю горячую перезагрузку (HMR).

Material UI (MUI)

– Скорость разработки интерфейса: предоставляет богатый набор готовых, доступных и адаптируемых UI-компонентов (Grid-сетка, диалоговые окна, формы, иконки, кнопки).

– Адаптивность «из коробки»: компоненты корректно отображаются на десктопах, планшетах и мобильных устройствах.

– Единая система стилизации: простая настройка темы (Theme Provider), включение темного/светлого режима оформления.

3. Синергия стека и преимущества для командной разработки

– Разделение ответственности (Decoupling): архитектура с чётким разделением на REST API (ASP.NET Core) и SPA (React) позволяет вести независимую параллельную разработку: один член команды может заниматься бизнес-логикой и БД, второй – вёрсткой и клиентом

– Сквозная строгая типизация (End-to-End Type Safety): использование C# на бэкенде и TypeScript на фронтенде сводит к минимуму несоответствие типов данных между клиентом и сервером.

– Соответствие индустриальным стандартам: данная комбинация технологий является одной из самых востребованных и актуальных в современной коммерческой веб-разработке.

Итоговая схема программной архитектуры проекта приведена на рисунке 3.

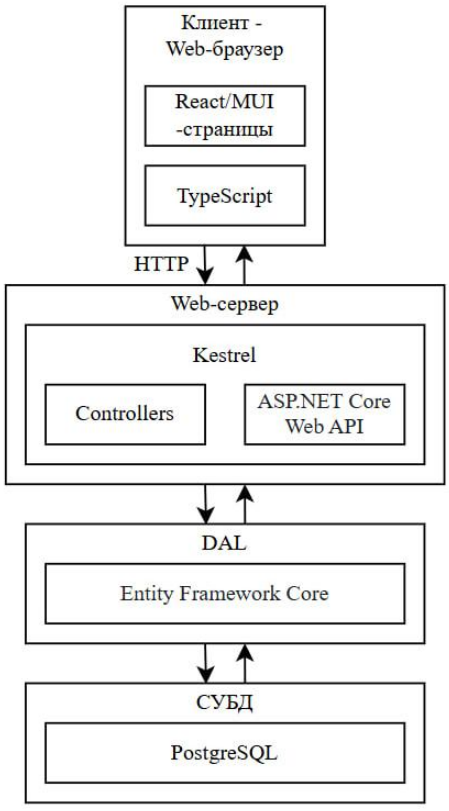


Рисунок 3 – программная архитектура проекта

4. Инициализация и первичная настройка проектов

1) Создание стартового проекта Backend (ASP.NET Core)

– Сгенерирован базовый каркас REST API приложения на .NET;

– Настроена конфигурация встроенного веб-сервера Kestrel;

– Подключены библиотеки Entity Framework Core для взаимодействия с СУБД PostgreSQL;

– Задана базовая структура сервисов, контроллеров и моделей данных.

2) Создание стартового проекта Frontend (React + TypeScript)

– Развёрнута заготовка Single Page Application (SPA) с использованием сборщика Vite и типизации TypeScript;

– Подключена и настроена библиотека UI-компонентов Material UI (MUI);

– Сформирована базовая маршрутизация и структура компонентов клиентской части.

Макеты отдельных страниц веб-приложения разработаны на основе спроектированной информационной структуры и представлены на рисунках 4-7. При создании макетов применялись специализированный инструмент – Visio.

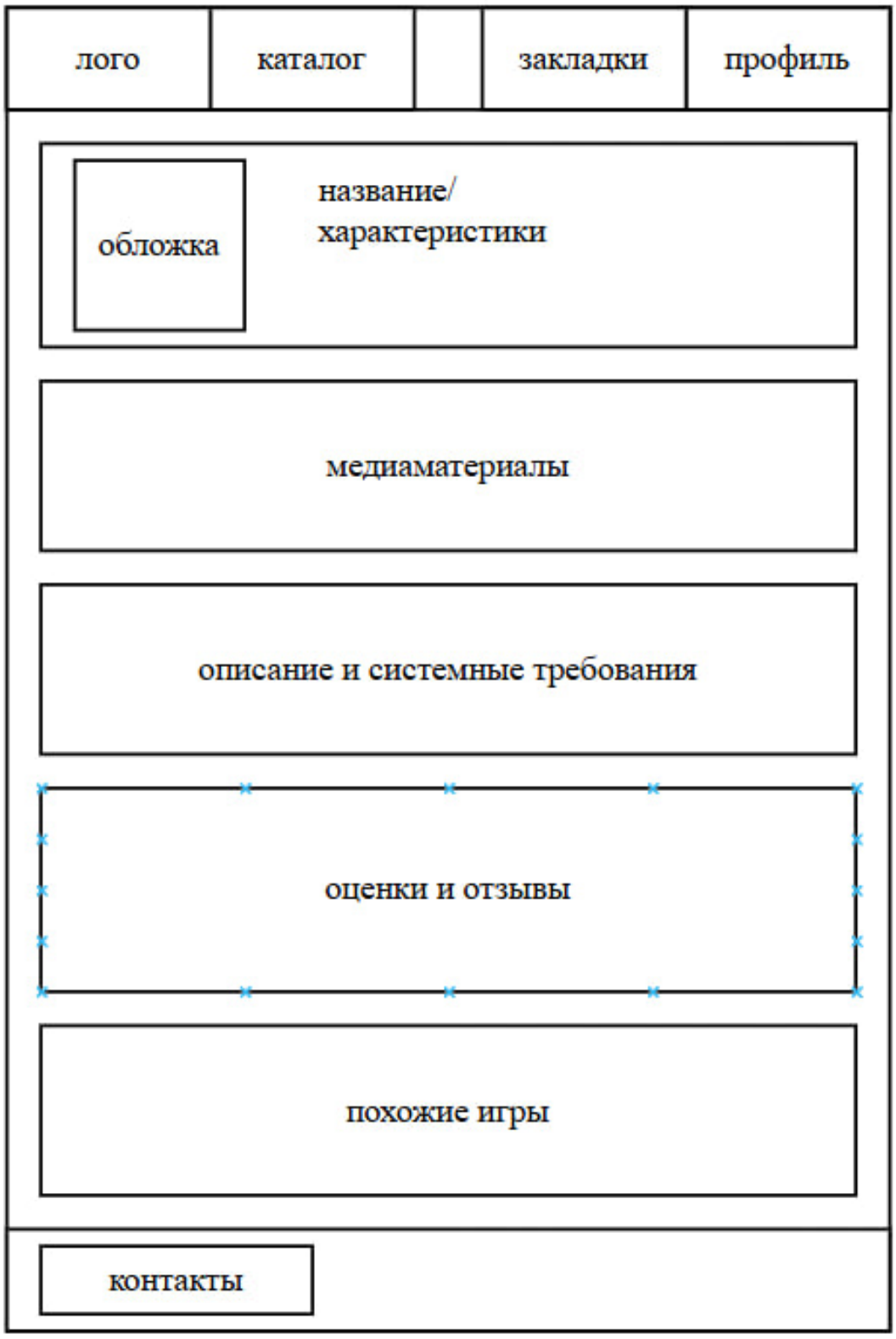


Рисунок 4 – Макет страницы игры

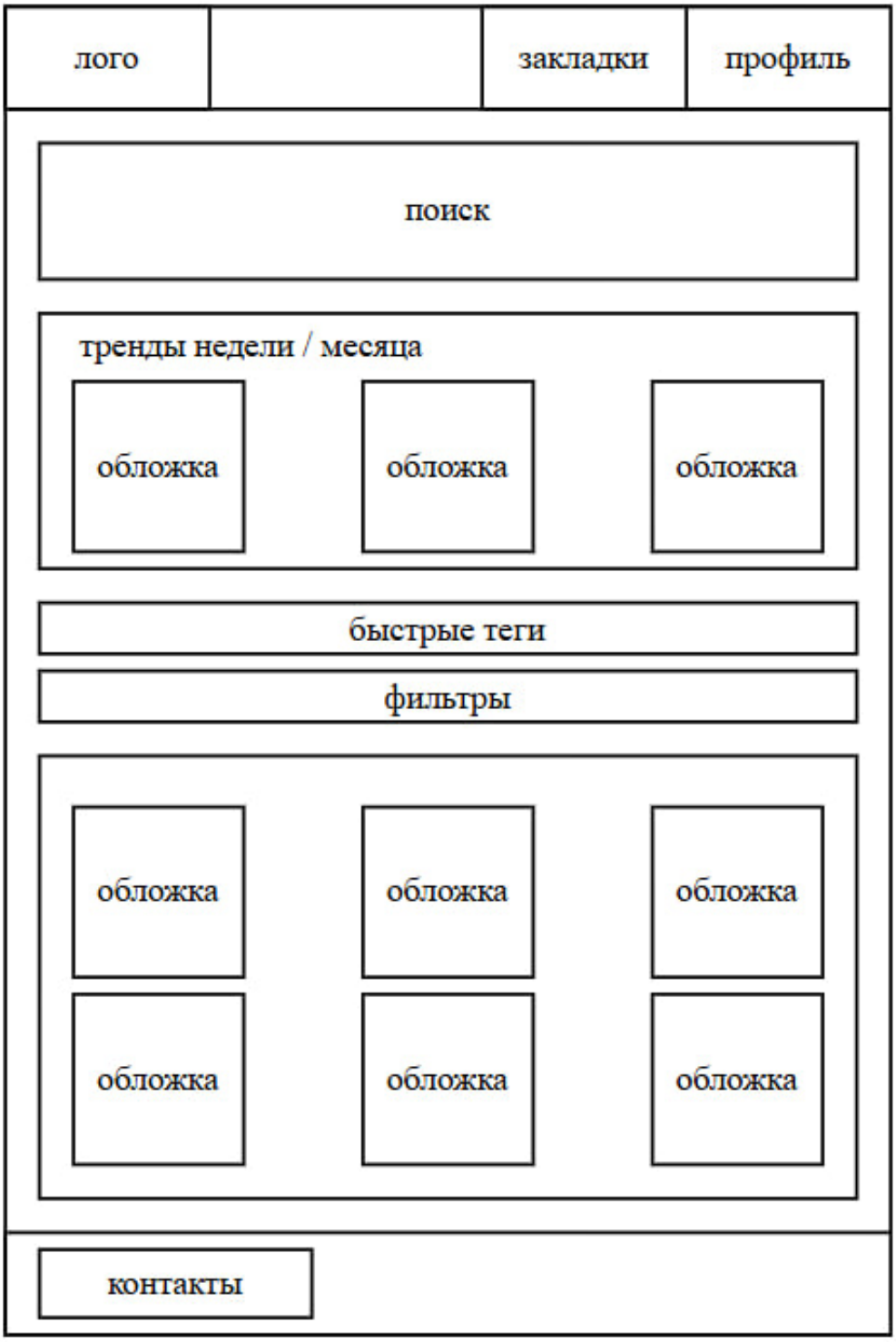


Рисунок 5 – Макет главной страницы

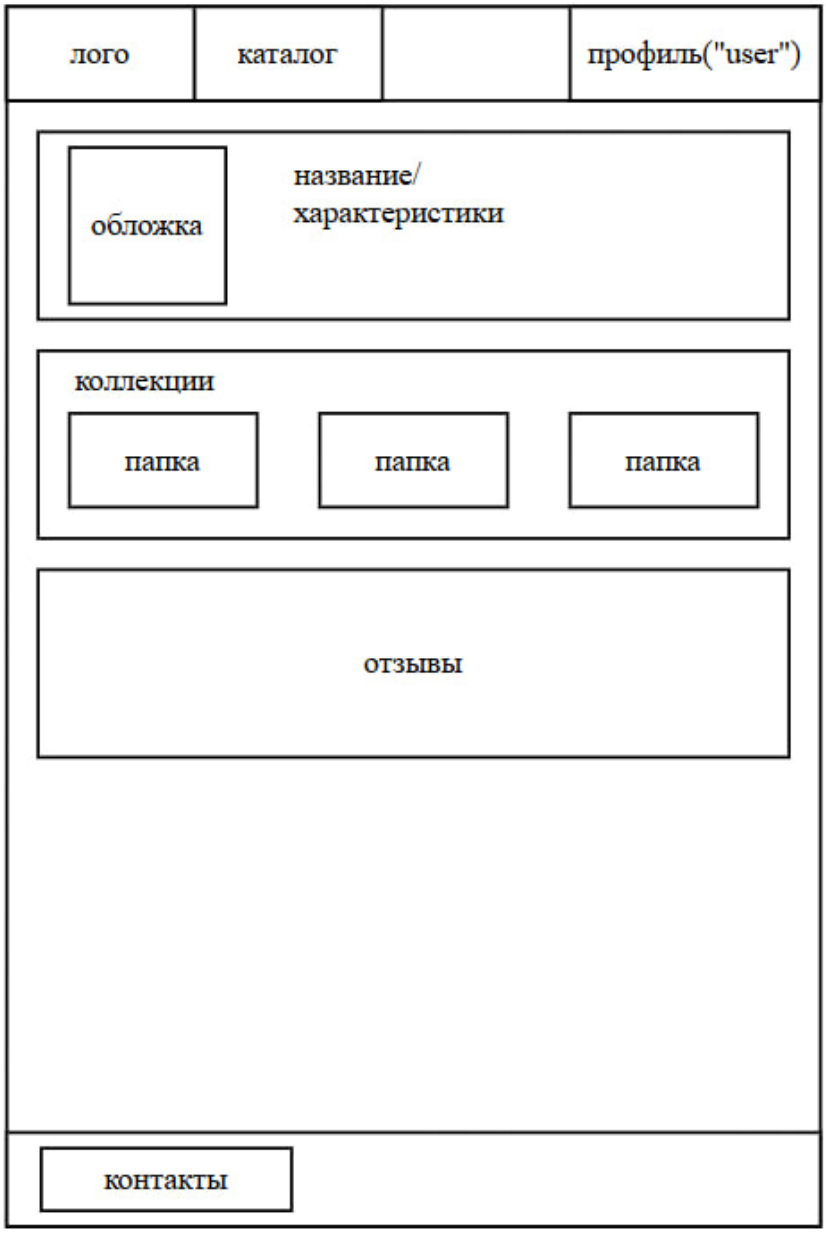


Рисунок 6 – Макет профиля

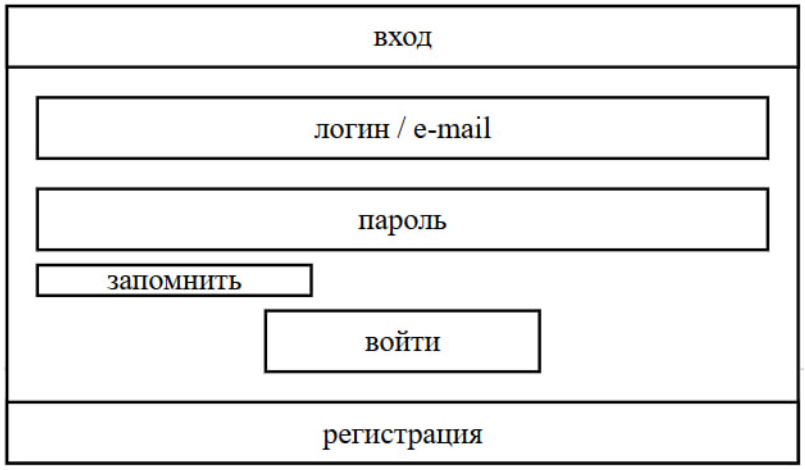


Рисунок 7 – Макет окна входа в профиль

После обсуждения проекта с преподавателем были обнаружены некоторые неточности и утверждены новые технические спецификации (рисунок 8).

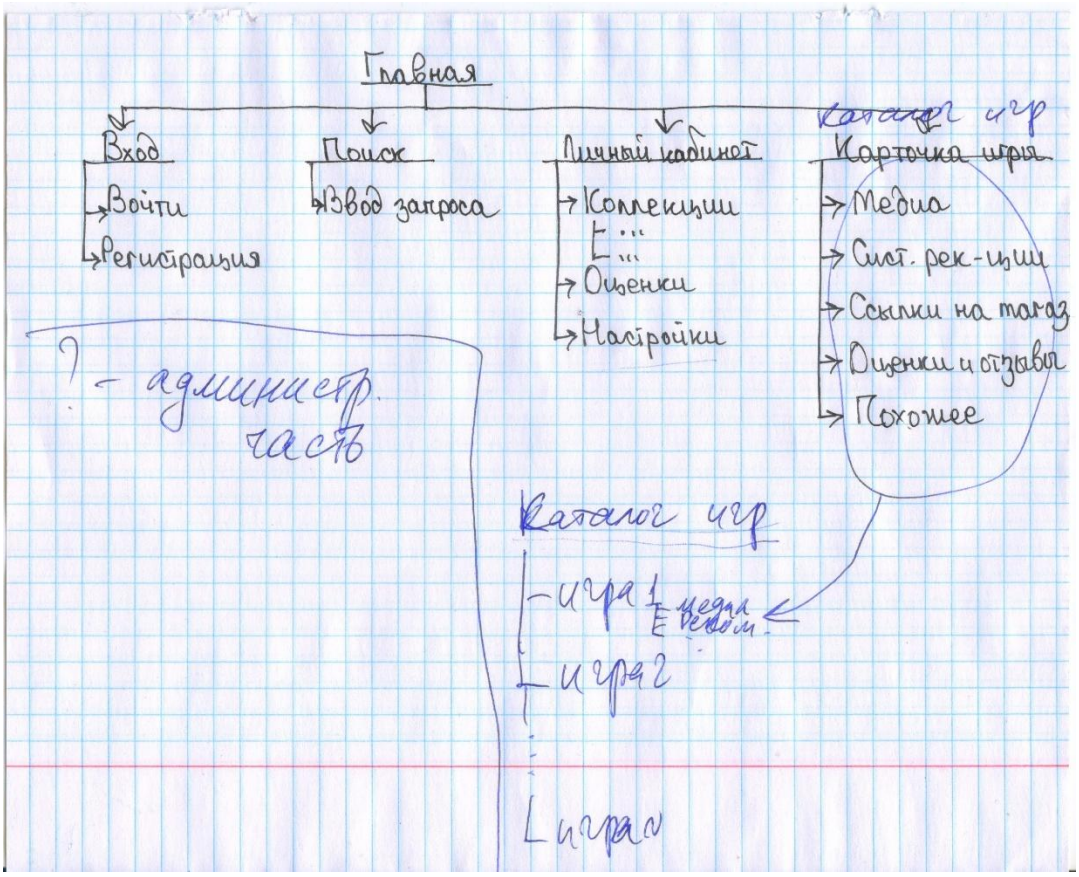


Рисунок 8 – Согласованные спецификации

Для их исправления были внесены определенные поправки:

1. Спроектирована самостоятельная информационная структура панели администратора (рисунок 9), функционирующая как отдельный закрытый ресурс. Данный модуль включает в себя задачи по:

– управлению учётными записями пользователей (просмотр списка, блокировка, разблокировка, удаление);

– модерации пользовательского контента (просмотр и удаление отзывов, комментариев, обработка жалоб);

– администрированию каталога видеоигр (добавление новых позиций, редактирование, удаление и изменение статуса видимости карточек).

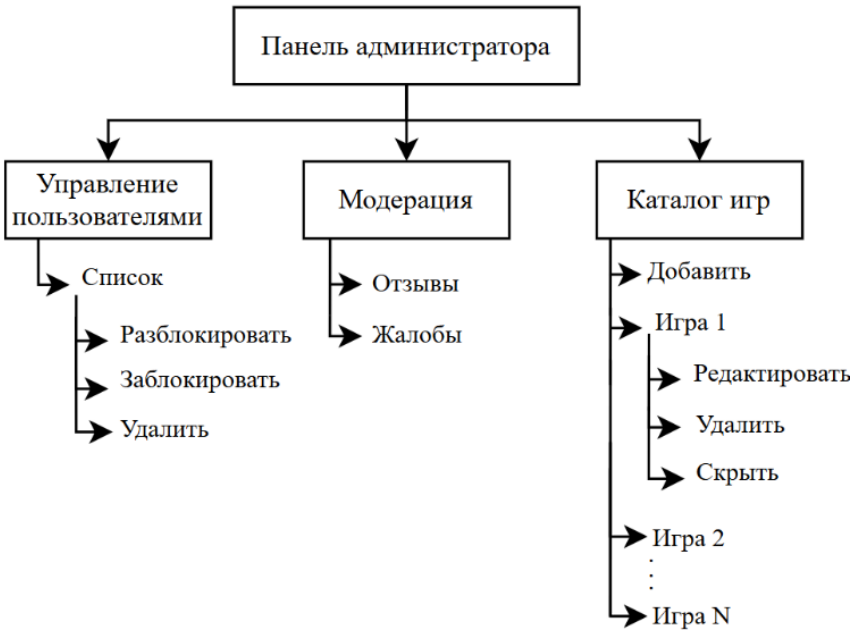


Рисунок 9 – Информационная структура модуля администратора

UML-диаграмма тоже изменилась (рисунок 10):

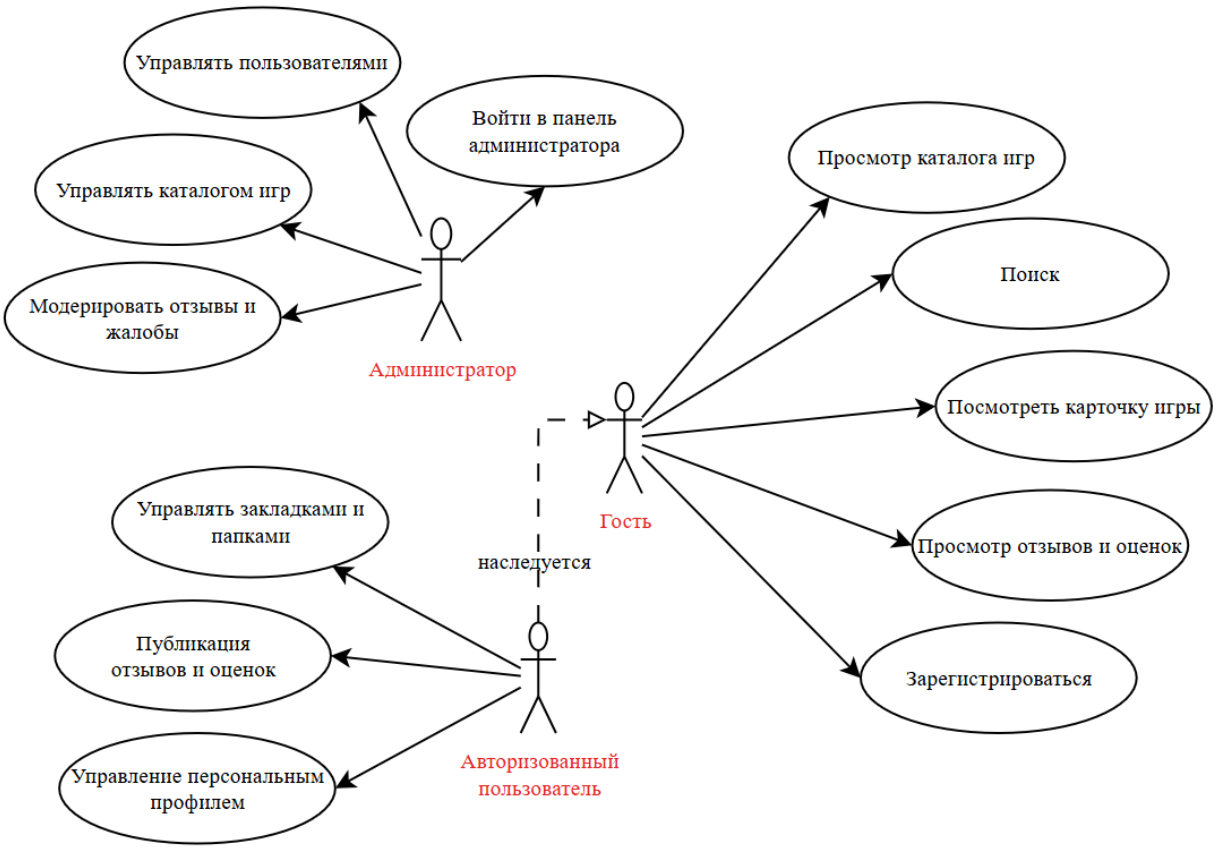


Рисунок 10 – Обновленная uml-диаграмма

2. Также была немного доработана информационная структура основного рисунка (рисунок 11).

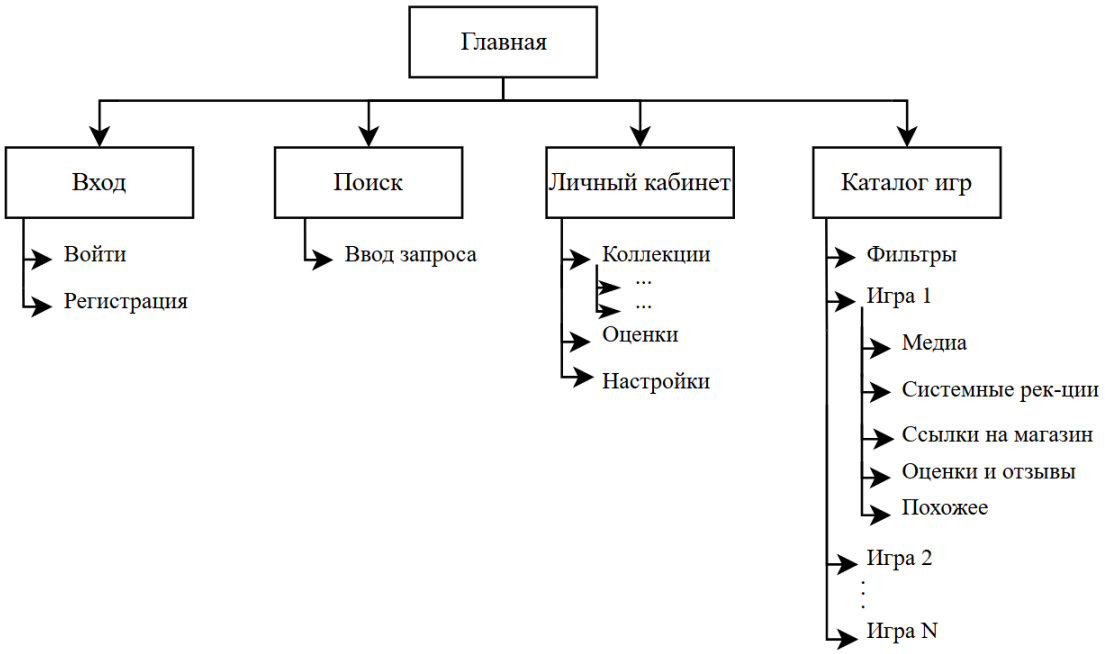


Рисунок 11 – Обновленная информационная структура

3. Для снижения информационной нагрузки на пользователя и оптимизации загрузки данных функционал каталога был вынесен из Главной страницы в самостоятельный раздел. На Главной странице оставлены витринные блоки («Новинки», «Популярные игры») с кнопкой переадресации «Показать все», ведущей на полноценную страницу «Каталог игр».

4. Сложная фильтрация (по жанрам, годам выпуска, платформам) и сортировка перенесены со страницы приветствия непосредственно в раздел «Каталог игр».

Были спроектированы макеты для модуля администратора (рисунки 12-15).

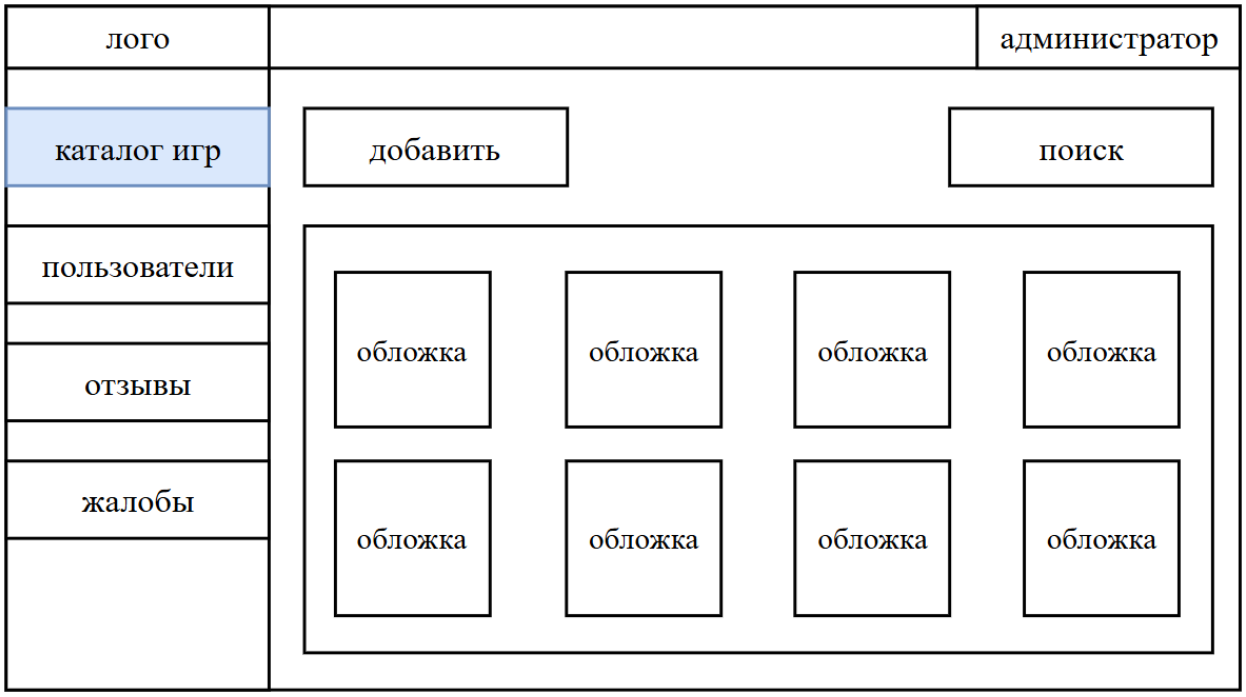


Рисунок 12 – Модуль администратора. каталог игр

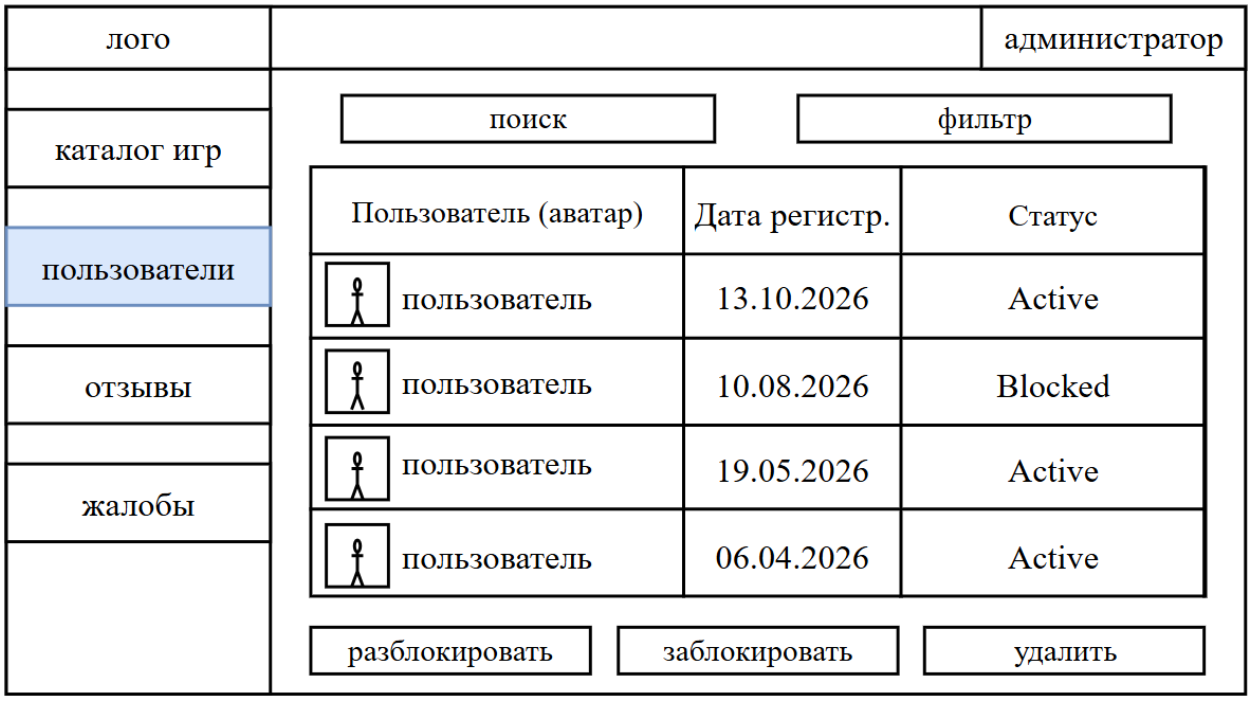


Рисунок 13 – Модуль администратора. пользователи



Рисунок 14 – Модуль администратора. отзывы

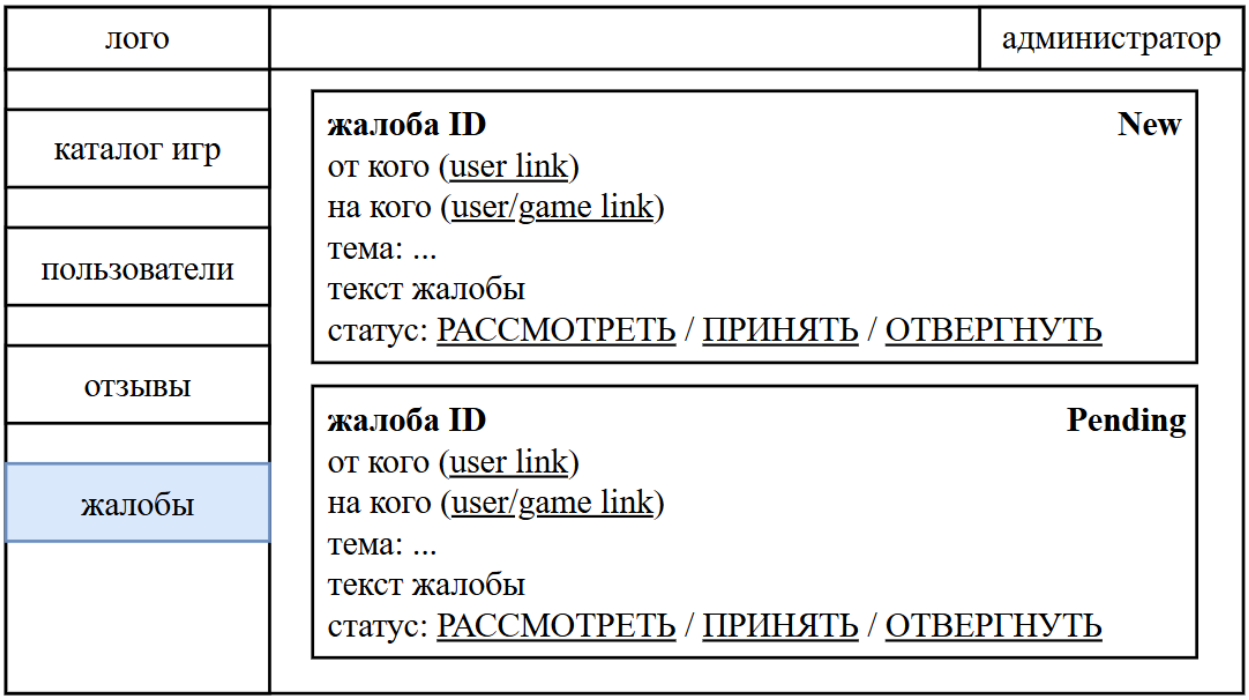


Рисунок 15 – Модуль администратора. жалобы

Вывод: в ходе выполнения лабораторной работы было успешно развёрнуто и настроено рабочее пространство для командной разработки веб-приложения «Игровая энциклопедия» (сформирована структура репозитория Git, файлы конфигурации .gitignore и базовые технические спецификации в файле README.md).

В рамках проектирования системы были сформулированы функциональные требования для трёх категорий пользователей (Гость, Авторизованный пользователь, Администратор) и построена соответствующая UML-диаграмма прецедентов. Обоснован и выбран современный технологический стек: React, TypeScript, Vite, Material UI на клиентской стороне и ASP.NET Core Web API, EF Core, PostgreSQL на серверной стороне, обеспечивающий сквозную строгую типизацию и высокую производительность.

По результатам рецензирования проекта и согласования спецификаций с преподавателем была проведена существенная оптимизация информационной архитектуры приложения:

1. Осуществлено разделение публичного каталога и Главной страницы с переносом тяжелых механизмов фильтрации и постраничной навигации в самостоятельный раздел «Каталог игр», что позволило снизить когнитивную и вычислительную нагрузку на приветственную страницу.

2. Проектирование служебного функционала было вынесено в изолированный модуль – «Панель администратора», функционирующий как отдельный закрытый ресурс. Для него разработана собственная информационная структура, включающая задачи администрирования пользователей, модерации отзывов/жалоб и CRUD-операций с каталогом видеоигр.

3. Разработан полный комплект низкодетализированных макетов (Wireframes) как для публичной части приложения (Главная, Каталог, Профиль, Вход, Карточка игры), так и для всех экранов административного модуля.

Все спроектированные схемы, спецификации и исходные заготовки проектов сохранены в репозитории.